

Healthcare RAG Assistant for Skin Disease Information

Submitted By:

Name	ID
Aseel Alammari	
Dhai Alshareef	
Maysim Badhurays	
Dalia Babbain	

Contents

1. Introduction.....	3
2. Problem Description	3
3. Proposed Solution.....	3
4. System Architecture.....	4
5. RAG Components.....	4
6. Dataset Description.....	6
7. Evaluation Methodology	7
7.1. Metrics Definitions.....	7
7.2. Results	8
7.3. Analysis of Results	8
8. Prototype Interface and Deployment Description	9
8.1. Access Instructions:.....	9
8.2. Deployment Requirements	9
9. Conclusion	11
10. Team Responsibilities	12
11. References.....	13

1. Introduction

Access to reliable medical information about skin conditions remains a challenge for many users. This project presents a Healthcare RAG Assistant designed to answer questions about four common skin diseases: acne, eczema, psoriasis, and skin cancer.

The system uses trusted healthcare documents from sources such as CDC and MedlinePlus as its knowledge base. The documents are processed into embeddings and stored for semantic retrieval. When a user submits a question, the system retrieves the most relevant information and uses a generative AI model to produce accurate responses based on the retrieved content.

The project demonstrates an end-to-end Retrieval-Augmented Generation (RAG) workflow, including document processing, retrieval, answer generation, and user interaction through a Streamlit interface.

2. Problem Description

Skin diseases are among the most prevalent health conditions worldwide. Many people turn to online sources to search for information about symptoms, causes, and treatments. However, not all online sources are reliable, and misinformation can lead to confusion or delayed medical care.

This project focuses on four common skin conditions: acne, eczema, psoriasis, and skin cancer. The core problem is the lack of a trustworthy and accessible tool that can answer users' medical questions based solely on verified sources. Therefore, there is a clear need for a system that retrieves accurate information from trusted healthcare documents and presents it in a clear and understandable way.

3. Proposed Solution

To address the identified problem, this project proposes a Retrieval-Augmented Generation (RAG) healthcare assistant for skin disease information. The system is designed to answer users' questions about acne, eczema, psoriasis, and skin cancer using trusted medical documents from verified healthcare sources such as CDC, and MedlinePlus.

The system retrieves relevant information from the stored medical documents and then generates responses based on the retrieved content. This helps ensure that the answers are grounded in reliable sources rather than generating unsupported information. In addition, the prototype includes a user interface that allows users to enter questions and receive clear responses interactively.

4. System Architecture

The proposed system follows a Retrieval-Augmented Generation (RAG) architecture designed to answer users' questions about acne, eczema, psoriasis, and skin cancer. The system begins by collecting trusted medical documents related to these skin conditions. After preprocessing and embedding generation, the information is stored in a vector database to support efficient retrieval.

When a user submits a question through the Streamlit interface, the system retrieves the most relevant medical content from the stored documents and passes it to the language model to generate a grounded response. Finally, the generated answer is displayed to the user through the interface.

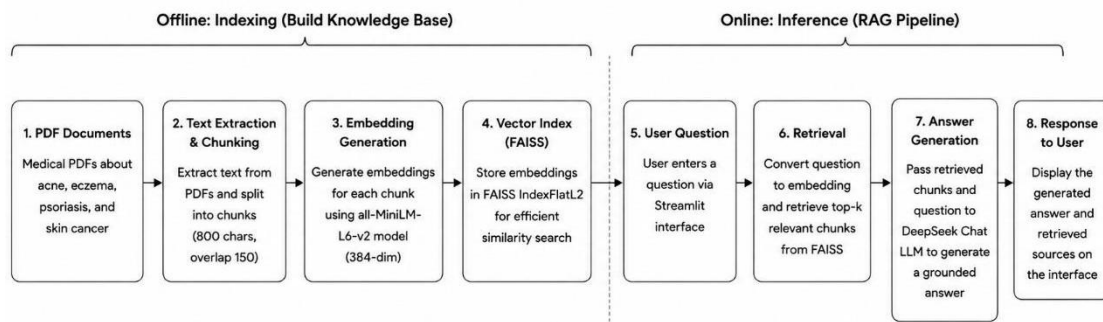


Fig. 1. Overall architecture of the proposed Healthcare RAG Assistant system.

5. RAG Components

The system is composed of five main RAG components. Each component has a specific role in transforming the medical PDF documents into clear answers for the user.

1. Document Ingestion

The first step is document ingestion. In this stage, the system loads the medical PDF files related to acne, eczema, psoriasis, and skin cancer. The pypdf library is used to read each PDF file and extract the text from every page. This allows the system to convert the medical documents from PDF format into raw text that can be processed later.

2. Text Chunking

After extracting the text, the system divides it into smaller text chunks. This is done because long documents are difficult to search and process as one large text. The system uses a sliding window approach, where each chunk contains about 800 characters with an overlap of 150 characters. The overlap helps preserve context between chunks so that important information is not lost between sections. This process resulted in a total of 54 chunks across all four documents.

3. Embedding Generation

The next step is embedding generation. Each text chunk is converted into a numerical vector using the all-MiniLM-L6-v2 sentence transformer model. These vectors, called embeddings, represent the meaning of the text in a format that the system can compare and search efficiently. In this project, each chunk is represented as a 384-dimensional vector.

4. Vector Indexing and Retrieval

After generating the embeddings, the chunk embeddings are first converted to float32 and normalized using L2 normalization before being stored in a FAISS IndexFlatL2 vector index. FAISS is used to make the search process faster and more efficient. When the user enters a question, the system converts the question into an embedding using the same embedding model. The query embedding is also converted to float32 and normalized using L2 normalization before searching. Then, FAISS compares the normalized question embedding with the normalized stored chunk embeddings and retrieves the most relevant chunks using similarity search. The system retrieves the top 5 most relevant chunks for each user question.

5. Answer Generation and Interface

The retrieved chunks are passed to the DeepSeek Chat language model through the OpenAI SDK. The model generates the final response using a temperature of 0.0 and a maximum of 500 tokens, which help produce concise and controlled answers. A structured prompt is used to guide the model to answer only from the retrieved context, avoid diagnosis, not replace professional medical advice, and keep the response clear and concise.

The prompt used to control the model response is shown below:

```
prompt = f"""
You are a healthcare information assistant.

Answer the question using ONLY the provided context.
If the answer is not found in the context, say:
"I could not find enough information in the provided documents."

Rules:
- Do not diagnose the user.
- Do not replace professional medical advice.
- Keep the answer clear and concise.
- Do not include unrelated information.
```

Finally, the generated answer is displayed to the user through the Streamlit interface, where the user can type questions, view the system's response, and check the retrieved document sources.

6. Dataset Description

The dataset used in this project consists of four trusted medical PDF documents related to acne, eczema, psoriasis, and skin cancer. These documents were collected from reliable healthcare sources and used as the knowledge base for the RAG system.

Acne is covered through a MedlinePlus document that explains acne as a common skin condition that causes pimples when hair follicles become clogged. It also discusses possible causes, affected areas, and basic treatment options [1].

Eczema is covered through a MedlinePlus document that describes eczema, also called dermatitis, as a condition that can cause dry, itchy skin and rashes. It also mentions that eczema may be linked to genetic and environmental factors [2].

Psoriasis is covered through a MedlinePlus document that explains psoriasis as a skin disease that causes itchy or sore patches of thick, red skin with silvery scales. It also describes its relation to the immune system and long-term nature [3].

Skin cancer is covered through a CDC document that explains skin cancer as a disease where skin cells grow out of control. It also states that most cases are caused by overexposure to ultraviolet (UV) rays [4].

7. Evaluation Methodology

The system performance was evaluated using local RAG evaluation metrics. These metrics were selected to assess both main parts of the RAG pipeline: the retrieval component and the answer generation component.

The evaluation used:

- Test set: 20 curated questions related to acne, eczema, psoriasis, and skin cancer
- Retriever: FAISS IndexFlatL2 + all-MiniLM-L6-v2 embeddings
- Top-k: 5 retrieved chunks per question
- Generator: DeepSeek Chat
- Evaluation method: Local comparison using expected source, expected keywords, retrieved context, and reference answers

7.1. Metrics Definitions

- Context Precision: measures the proportion of retrieved chunks that came from the expected medical source.
- Context Recall: checks whether the expected source was retrieved within the top-k results.
- Context Entities Recall: measures how many expected medical keywords were found in the retrieved context.
- Noise Sensitivity measures: the proportion of irrelevant retrieved context. A lower score indicates better retrieval quality.
- Response Relevancy: measures how semantically relevant the generated answer is to the user's question.
- Faithfulness: measures how well the generated answer is supported by the retrieved medical context.
- Answer Correctness: compares the generated answer with the reference answer to evaluate answer accuracy.

7.2. Results

Metric	Score
Context Precision	0.99
Context Recall	1.00
Context Entities Recall	0.99
Noise Sensitivity	0.01
Response Relevancy	0.81
Faithfulness	0.71
Answer Correctness	0.93

Table 1. Local RAG evaluation results on the 20-question test set.

7.3. Analysis of Results

The retrieval results show strong performance across the 20-question test set. Context Precision scored 0.99, indicating that most retrieved contexts were relevant to the user questions. Context Recall achieved 1.00, showing that the system successfully retrieved the required medical information. Context Entities Recall scored 0.99, which means that almost all expected medical keywords were found in the retrieved context. In addition, Noise Sensitivity scored 0.01, indicating that only a very small amount of irrelevant information appeared in the retrieval results.

The generation results also show good answer quality. Answer Correctness scored 0.93, indicating that the generated answers were close to the reference answers. Response Relevancy scored 0.81 and Faithfulness scored 0.71, showing that the answers were generally relevant and supported by the retrieved medical context.

Overall, the results suggest that the system can retrieve useful medical information and generate accurate, context-based responses for the selected skin disease topics.

8. Prototype Interface and Deployment Description

The prototype was implemented using Streamlit to provide a simple and interactive web-based interface. The system allows users to ask medical questions related to acne, eczema, psoriasis, and skin cancer through a chat input field. After a question is submitted, the system retrieves the most relevant medical information from the stored PDF documents and generates a grounded response using the DeepSeek language model.

The interface was designed to be user-friendly and easy to use without requiring technical knowledge. It also displays the retrieved document sources used to generate the answer, which improves transparency and helps users identify the medical references used by the system.

The Healthcare RAG Assistant was deployed using Streamlit Community Cloud, providing a publicly accessible URL.

Source Code Repository: <https://github.com/Dhaialshareef/healthcare-rag-app/tree/main>

8.1. Access Instructions:

Application URL: <https://healthcare-rag-app-5fsqspsdgc7qdknpy3ng3x.streamlit.app/>

The deployed application can be accessed directly through a web browser without requiring installation or local setup.

To use the system:

- Open the application URL.
- Enter a medical question related to acne, eczema, psoriasis, or skin cancer.
- Submit the question through the interface.
- Receive a context-based response generated from trusted healthcare documents.

8.2. Deployment Requirements

- Web interface: Streamlit
- Embedding model: sentence-transformers/all-MiniLM-L6-v2
- Vector search index: FAISS-CPU
- Answer generation: DeepSeek API
- Knowledge source: Medical PDF documents stored in GitHub
- Python version: 3.10 or higher

When the application starts, it loads the medical PDF documents from the GitHub repository. The text is extracted from the documents, divided into smaller chunks, and converted into embeddings using the Sentence Transformer model. A FAISS index is then built and cached in memory to support fast document retrieval.

For each user query, the system searches the cached FAISS index to retrieve the most relevant document chunks. These chunks are sent together with the user's question to the DeepSeek API to generate a clear and context-based response. The API key is stored securely in Streamlit Secrets and is not included directly in the source code.

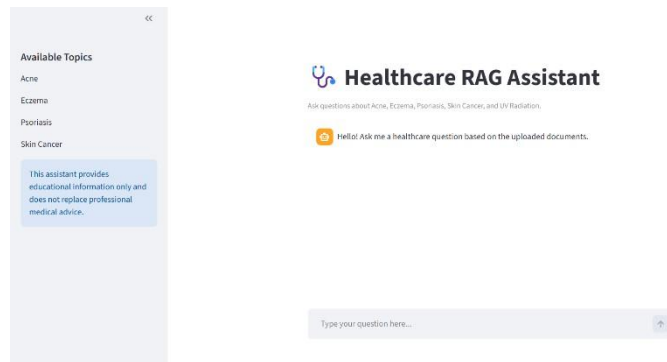


Fig. 2. Main interface of the Streamlit Healthcare RAG Assistant showing the supported skin disease topics and user input area.

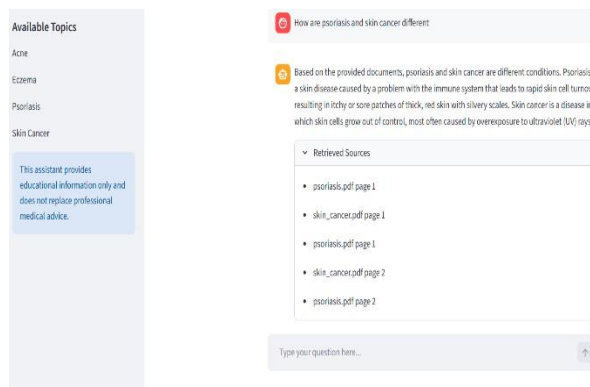


Fig. 3. Example interaction with the Healthcare RAG Assistant showing a generated response and retrieved medical document sources.

9. Conclusion

This project successfully demonstrates a complete Retrieval-Augmented Generation (RAG) system for the healthcare domain. The system provides reliable and traceable information about acne, eczema, psoriasis, and skin cancer by grounding its responses in trusted medical documents from CDC, and MedlinePlus.

The project satisfies the main requirements by using publicly available medical documents and integrating document processing, embedding generation, vector-based retrieval, and generative AI into one unified pipeline. In addition, the prototype was deployed online using Streamlit and tested through a user-friendly interface.

The evaluation results showed strong retrieval performance and good answer quality across the tested questions, demonstrating the system's ability to provide relevant medical responses grounded in trusted sources.

10. Team Responsibilities

Team Member	Responsibilities
Aseel Alammari	Collecting and preparing medical documents, processing PDF files and text chunking, contributing to the system pipeline development, and contributing to report writing.
Dhai Alshareef	Developing the Streamlit interface, handling deployment and GitHub integration, contributing to system integration, ensuring proper system functionality, and contributing to report writing.
Maysim Badhurays	Developing retrieval pipeline components (Embeddings and FAISS Retrieval), analyzing evaluation results and performance metrics, and contributing to report writing.
Dalia Babbain	Developing the answer generation component (DeepSeek Integration and Prompt Configuration), testing the system and validating output quality, and contributing to report writing.

11. References

[1] MedlinePlus, "Acne," *MedlinePlus*. [Online]. Available: <https://medlineplus.gov/acne.html>.

[Accessed: May 17, 2026].

[2] MedlinePlus, "Eczema," *MedlinePlus*. [Online]. Available: <https://medlineplus.gov/eczema.html>.

[Accessed: May 17, 2026].

[3] MedlinePlus, "Psoriasis," *MedlinePlus*. [Online]. Available: <https://medlineplus.gov/psoriasis.html>.

[Accessed: May 17, 2026].

[4] Centers for Disease Control and Prevention, "Skin Cancer Basics," *CDC*. [Online]. Available:

<https://www.cdc.gov/skin-cancer/about/index.html>. [Accessed: May 17, 2026].